



A JAX Ecosystem for Likelihood-Based Inference in HEP

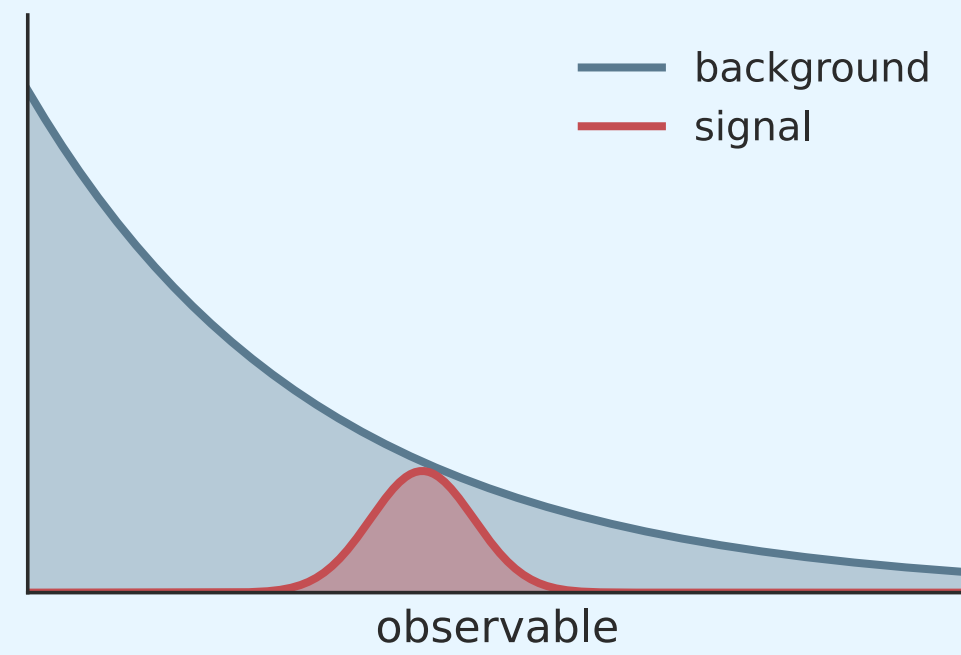
Mohamed Aly¹, Peter Fackeldey¹, Massimiliano Galli¹
Princeton University¹



Statistical Analyses in HEP

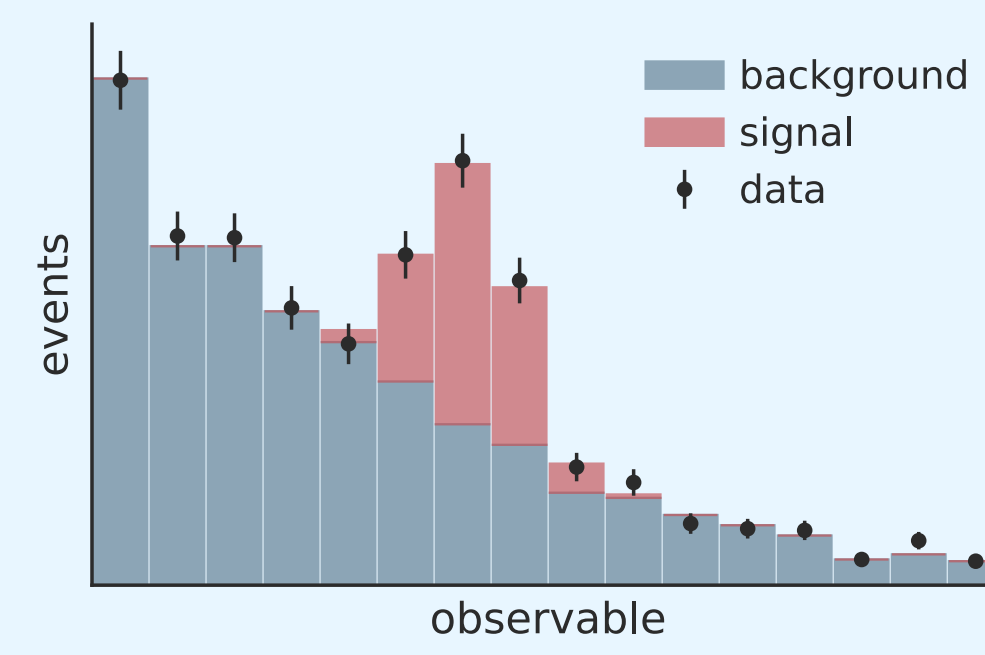
primitives

parameters . distributions . constraints



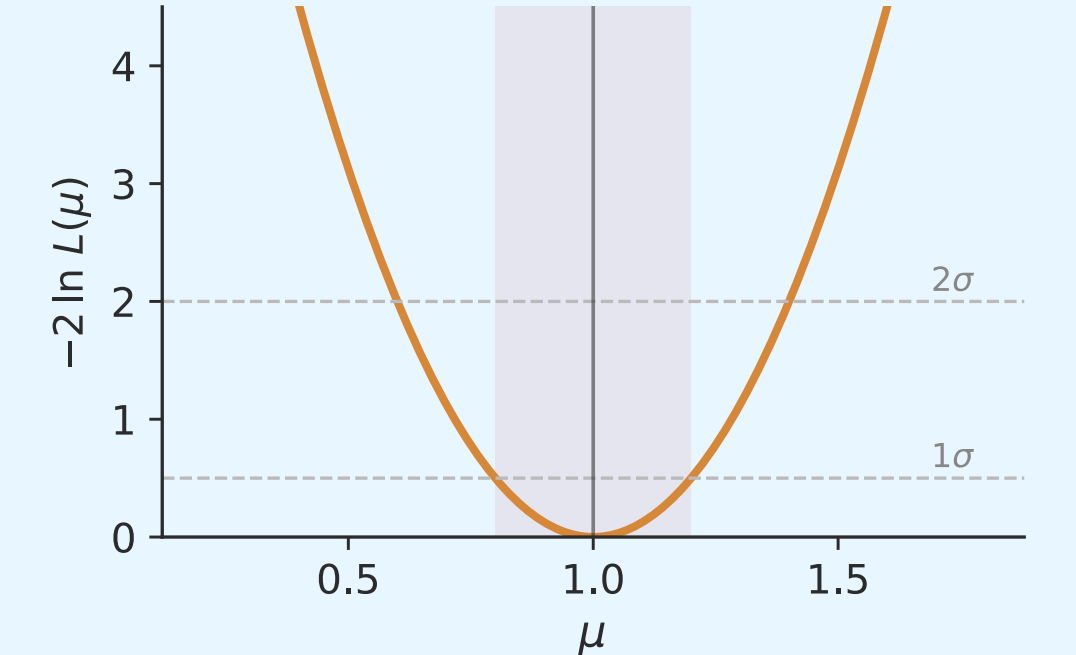
model

likelihoods . observations



inference

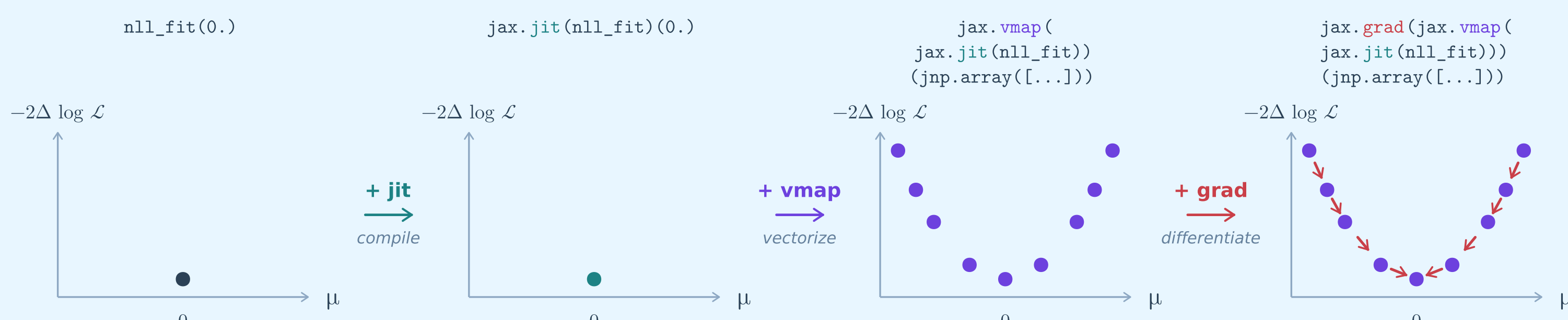
fits . intervals . limits



The case for JAX

Transforms

compile . vectorise . differentiate . compose



Ecosystem

machine learning integration . debugging . inspection

Machine Learning	flax . nnx networks in JAX
Optimisation	optimistix differentiable solvers
Dataclasses	equinox modules as PyTrees
Checkpointing	orbax serialisation of states
Inspection	mlflow . wandb . tensorboard logging . monitoring

A JAX ecosystem for HEP statistics

Primitives

```

1 import jax.numpy as jnp
2 import evermore as evm
3 hist = jnp.array([...])
4 syst = evm.NormalParameter(name="syst")
5 # lnN +/- 10% on the yield
6 modifier = syst.scale_log_asymmetric(
7     up=jnp.array([1.1]),
8     down=jnp.array([0.9]),
9 )
10 modified_hist = modifier(hist)

```

(parameters + constraints)

Quickstart

```

1 import paramore as pm
2 import evermore as evm
3 # parameters (a PyTree)
4 params = {
5     "sig_n": evm.Parameter(500.0),
6     "bkg_n": evm.Parameter(5000.0),
7     "mu": evm.Parameter(125.0),
8 }
9 # extended NLL from params + model
10 def nll(p, data):
11     return pm.create_extended_nll(p, model, data)

```

(unbinned)

```

1 import paramore as pm
2 lower, upper = 100.0, 180.0
3 sig = pm.Gaussian(mu=125.0, sigma=2.0,
4                   lower=lower, upper=upper)
5 bkg = pm.Exponential(lambd=0.05,
6                      lower=lower, upper=upper)
7 model = pm.SumPDF(
8     pdfs=[sig, bkg],
9     extended_vals=[500.0, 5000.0],
10    lower=lower, upper=upper,
11 )

```

(distributions)

```

1 from jax.random import normal, key
2 import everwillow as ew
3 import everwillow.statelib as sl
4 # observed unbinned masses
5 data = normal(key(0), (10_000,)) * 2 + 125
6 # fit & parameter uncertainties
7 state = sl.State.from_pytree(params)
8 obs = {"data": data}
9 result = ew.fit(nll, state, obs)
10 unc = ew.uncertainties(nll, result.params, obs)

```

(nll-agnostic)

Interoperability and likelihood combination

